Git-Grundlagen: Änderungen sicher nachvollziehen

Git und GitHub verwenden Begriffe, die ähnlich klingen und Verschiedenes bedeuten. Ein **Pull Request** ist nicht dasselbe wie `git pull`. Ein **Commit** ist nicht dasselbe wie ein Push. In dieser Lektion klärst du diese Begriffe an einem praktischen Ablauf: Du arbeitest in der GitHub-Weboberfläche, legst einen Branch an, committest eine Änderung, prüfst den Diff und startest eine geänderte Workflow-Datei manuell.

> Lernmaterial: CC BY 4.0 (siehe `LICENSE-CONTENT.txt`).

Was du lernst

- Dateien und Branches im eigenen Lern-Repository finden
- Änderungen über GitHub committen und den Diff kontrollieren
- einen Pull Request als Vorschlag zur Übernahme verstehen
- eine Workflow-Änderung auf einem Arbeitsbranch manuell ausführen

Git und GitHub: zwei verschiedene Dinge

Git ist ein Versionsverwaltungssystem. Es speichert den Projektverlauf als Folge von Commits; du kannst Git lokal auf deinem Rechner verwenden. **GitHub** ist ein Webdienst, der Git-Repositories hostet und Zusammenarbeit darum herum organisiert: Pull Requests, Issues und GitHub Actions. Für die GitHub-Weboberfläche brauchst du kein lokal installiertes Git.

In diesem Kurs arbeitest du in deinem eigenen Lern-Repository. Das Kurs-Repository enthält die Kursmaterialien und Vorlagen.

Ein **Repository** besteht aus den Projektdateien und dem vollständigen Verlauf ihrer Änderungen. Öffne dein Lern-Repository auf GitHub und wähle den Tab **Code**. Dort siehst du die Dateien, darunter die `README.md`. Die Branch-Auswahl zeigt, welcher Branch gerade angezeigt wird. Der **Standardbranch** ist der Hauptbranch des Repositorys, meist `main`.

Wie du das Lern-Repository einrichtest und Dateien im Browser speicherst, steht in Teil 1.

Commits und Branches

Ein **Commit** hält einen Stand des Projekts fest. Technisch speichert Git dabei einen Snapshot der Dateien, nicht die Änderung selbst. Den **Diff**, also die Gegenüberstellung zweier Stände, berechnet Git bei Bedarf daraus.

Ein **Branch** ist ein benannter Zeiger auf einen Commit. Legst du neue Commits an, wandert der Zeiger des aktiven Branches mit. Darum sind Branches billig: Ein neuer Branch kopiert keine Dateien, er legt nur einen weiteren Zeiger an. So kannst du Änderungen vorbereiten, ohne den Standardbranch zu berühren.

In der GitHub-Oberfläche bearbeitest du eine Datei und speicherst mit **Commit changes**. Die Weboberfläche erstellt den Commit direkt auf dem gewählten Branch. Eine gute Commit-Nachricht beschreibt die Änderung, zum Beispiel `Ergänze YAML-Schrittbeschreibung`.

Beim lokalen Arbeiten gibt es einen Zwischenschritt: das **Staging**. Dort wählst du aus, welche Änderungen in den nächsten Commit kommen. Die Weboberfläche zeigt diesen Schritt nicht separat. Diese Lektion behandelt nur den Browserweg.

Übung: Arbeitsbranch anlegen

1. Öffne im Repository den Tab **Code** und die Branch-Auswahl.
2. Gib einen neuen Branch-Namen ein.
3. Wähle die Option, den Branch vom Standardbranch aus zu erstellen.
4. Wechsle in den neuen Branch.

Die Beschriftungen der Oberfläche können sich ändern. Die aktuelle Anleitung steht in der GitHub-Dokumentation.[1]

Kontrolle: Die Branch-Auswahl zeigt deinen neuen Namen als aktuellen Branch. Prüfe vor jeder Bearbeitung beides: Repository und Branch.

Remote, Push und Pull

Ein **Remote** ist ein anderes Git-Repository, mit dem dein lokales Repository Änderungen austauscht. In diesem Kurs ist das dein Lern-Repository auf GitHub.

[1] GitHub-Dokumentation zum Erstellen von Branches über die Weboberfläche, geprüft am 7. November 2026: https://docs.github.com/en/pull-requests/how-tos/commit-changes/managing-branches-within-your-repository[2]

- **Push** überträgt lokale Commits zum Remote.
- **Pull** holt Änderungen vom Remote in dein lokales Repository.

Wenn du im Browser committest, arbeitest du direkt im Remote. Einen Push gibt es dort nicht, weil nichts lokal vorliegt.

Pull Request: Änderung zur Prüfung vorschlagen

Ein **Pull Request** (PR) ist ein Vorschlag, Änderungen aus einem Branch in einen anderen zu übernehmen. Er hat mit `git pull` nichts zu tun. Beteiligte können die Änderungen ansehen, kommentieren und ein Review abgeben. Ob eine Freigabe Voraussetzung für den Merge ist, hängt von den Repository- oder Branch-Regeln ab. Erst der **Merge** übernimmt die Änderungen in den Zielbranch. GitHub bietet dafür mehrere Varianten an (Merge-Commit, Squash und Rebase).[3] Für diese Lektion genügt die Standardvariante.

Übung: Pull Request erstellen

1. Ändere auf deinem Arbeitsbranch eine Datei und speichere mit **Commit changes**.
2. Öffne **Pull requests** und erstelle einen neuen PR. Quellbranch ist dein Arbeitsbranch, Zielbranch der Standardbranch.
3. Prüfe die angezeigten Branch-Namen, bevor du den PR anlegst.
4. Öffne **Files changed** und kontrolliere, dass nur die beabsichtigte Änderung enthalten ist.

Ein erstellter PR übernimmt nichts. Merge den PR nur, wenn die Aufgabe es verlangt. Manche Aufgaben sehen eine direkte Änderung auf dem Standardbranch vor, etwa die `README.md` in Markdown-Grundlagen. Dann gilt die Aufgabenstellung.

Workflow-Dateien und GitHub Actions

Eine **Workflow-Datei** liegt als YAML-Datei im Repository unter `.github/workflows/`. **GitHub Actions** führt Workflows aus, sobald ihr **Trigger** eintritt, also das Ereignis, das den Workflow startet. Der Trigger `workflow_dispatch` ermöglicht den manuellen Start.

Zwei Regeln sind für die folgende Übung entscheidend:

[3] GitHub-Dokumentation zu Merge-Commit, Squash und Rebase, geprüft am 7. November 2026: https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/incorporating-changes-from-a-pull-request/about-pull-request-merges[4]

- **Run workflow** wird für einen Workflow mit `workflow_dispatch` angeboten, wenn dessen Datei auf dem **Standardbranch** liegt.[5]
- Beim Start wählst du einen Branch aus. GitHub führt dann die Version der Workflow-Datei aus, die auf diesem Branch liegt. So testest du Änderungen, ohne sie vorher in den Standardbranch zu übernehmen.[7]

Brückenübung: Workflow-Änderung prüfen

Ziel: Du änderst auf einem Arbeitsbranch den Namen eines Workflow-Schritts, prüfst den Diff und siehst die Änderung in einem manuellen Lauf.

Voraussetzungen: - Die Datei `.github/workflows/01-hallo-linux.yml` liegt auf dem Standardbranch deines Lern-Repositorys und enthält `workflow_dispatch`. - Du hast einen Arbeitsbranch, der vom aktuellen Standardbranch abzweigt. Prüfe in der Dateiansicht, dass die Workflow-Datei dort ebenfalls vorhanden ist.

Schritte:

1. Wechsle auf deinen Arbeitsbranch und öffne `.github/workflows/01-hallo-linux.yml`.
2. Ändere nur den Wert bei `name:` des Schritts `Begrüßung`, zum Beispiel in `Begrüßung aus meinem Arbeitsbranch`. Lass die Einrückung unverändert. Der Wert `name` ändert nur die Beschriftung des Schritts, nicht den ausgeführten Befehl.
3. Speichere mit **Commit changes** auf dem Arbeitsbranch, etwa mit der Nachricht `Benenne Begrüßungsschritt um`.
4. Öffne den Commit und prüfe den Diff. Geändert sein darf nur der `name:`-Wert.
5. Öffne **Actions**, wähle **01 - Hallo Linux** und klicke **Run workflow**. Wähle im Feld **Branch** deinen Arbeitsbranch und starte den Lauf.
6. Öffne den Lauf und den Job `erkunden`. Der Schritt sollte den neuen Namen tragen und weiterhin `echo "Hallo aus dem Linux-Runner!"` ausführen.

Kontrolle: - *Diff:* Trigger, Job und `run`-Befehl sind unverändert. - *Lauf:* Der Schrittname im Job entspricht deiner Änderung.

Typische Fehler:

[5] GitHub-Dokumentation zum manuellen Start: Die Workflow-Datei muss auf dem Standardbranch liegen; beim Start lässt sich der auszuführende Branch auswählen. Geprüft am 7. November 2026: https://docs.github.com/en/actions/how-tos/manage-workflow-runs/manually-run-a-workflow[6]

[7] GitHub-Dokumentation zum manuellen Start: Die Workflow-Datei muss auf dem Standardbranch liegen; beim Start lässt sich der auszuführende Branch auswählen. Geprüft am 7. November 2026: https://docs.github.com/en/actions/how-tos/manage-workflow-runs/manually-run-a-workflow[8]

Symptom	Wahrscheinliche Ursache
YAML-Fehler oder Schritt fehlt	Einrückung verändert oder Doppelpunkt nach `name` gelöscht
Name enthält einen Doppelpunkt	Wert in Anführungszeichen setzen
Lauf zeigt den alten Namen	Beim Start war ein anderer Branch gewählt, oder der Commit liegt nicht auf dem Arbeitsbranch
Änderung fehlt im Diff	Nicht mit **Commit changes** gespeichert
Datei fehlt im Arbeitsbranch	Branch war zu alt oder falsch angelegt; neuen Branch vom aktuellen Standardbranch erstellen
Run workflow fehlt	Datei mit `workflow_dispatch` liegt nicht auf dem Standardbranch

Rückgängig machen: Committe die Datei auf dem Arbeitsbranch mit dem ursprünglichen Namen zurück, oder lösche den Arbeitsbranch, solange du ihn nicht gemergt hast. Auf den Standardbranch hat die Übung keinen Einfluss.

Optional kannst du anschließend einen Pull Request öffnen. Merge ihn nur, wenn die Aufgabe es verlangt.

Vor dem Speichern prüfen

- Ist das richtige Repository geöffnet?
- Ist der richtige Branch ausgewählt?
- Zeigt der Diff nur die beabsichtigten Änderungen?
- Beschreibt die Commit-Nachricht die Änderung?

Übe zunächst an kleinen Änderungen. Befehle, die Verlauf umschreiben oder Arbeit verwerfen, brauchst du in dieser Lektion nicht. Zwei Beispiele für spätere Lektionen: `git reset --hard` verwirft ungesicherte lokale Änderungen, `git push --force` überschreibt veröffentlichten Verlauf.

Weiterlernen

- GitHub-Dokumentation: Branches verwalten[9]
- GitHub-Dokumentation: Pull Requests erstellen[10]
- GitHub-Dokumentation: Workflow manuell starten[11]
- GitHub-Dokumentation: Laufhistorie ansehen[12]
- *Pro Git*: Git-Grundlagen[13] und Branches[14]

Glossar

- **Branch:** Benannter Zeiger auf einen Commit, der mit neuen Commits weiterwandert.
- **Commit:** Gespeicherter Snapshot des Projekts.
- **Diff:** Gegenüberstellung zweier Stände.
- **Git:** Verteiltes Versionsverwaltungssystem, das auch lokal verwendet werden kann.
- **GitHub:** Webdienst zum Hosten von Git-Repositories und für die Zusammenarbeit.
- **GitHub Actions:** GitHub-Dienst, der Workflows anhand ihrer Konfiguration ausführt.
- **Merge:** Zusammenführen von Änderungen aus einem Branch in einen anderen.
- **Pull:** Änderungen vom Remote in ein lokales Repository holen.
- **Pull Request (PR):** Vorschlag, Änderungen aus einem Branch zu prüfen und zu übernehmen.
- **Push:** Lokale Commits zum Remote übertragen.
- **Remote:** Anderes Repository, mit dem Änderungen ausgetauscht werden.
- **Repository:** Projektdateien samt Git-Verlauf.
- **Staging:** Lokale Auswahl der Änderungen für den nächsten Commit.
- **Standardbranch:** Hauptbranch eines Repositorys, meist `main`.
- **Trigger:** Ereignis, das einen Workflow startet.
- **Workflow-Datei:** YAML-Datei mit der Konfiguration eines GitHub-Actions-Workflows.
- **`workflow_dispatch`:** Trigger für den manuellen Workflow-Start.

Schlagworte

`Git` `GitHub` `GitHub Actions` `Branch` `Commit` `Pull Request` `Workflow` `Versionsverwaltung`

[9] https://docs.github.com/en/pull-requests/how-tos/commit-changes/managing-branches-within-your-repository
[10] https://docs.github.com/en/pull-requests/how-tos/create-pull-requests
[11] https://docs.github.com/en/actions/how-tos/manage-workflow-runs/manually-run-a-workflow
[12] https://docs.github.com/en/actions/how-tos/monitor-workflows/viewing-workflow-run-history
[13] https://git-scm.com/book/en/v2/Git-Basics-Getting-a-Git-Repository
[14] https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell